

- EXAMEN DE 1^{ère} SESSION -

NOM (en majuscules) :

Date : 11/04/2016

Prénom :

Section / Année :

Local :

UE : Principes d'algorithmique et programmation – C#

Epreuve : Examen final

Professeur : François Koeune

-
- Matériel autorisé : PC, Visual Studio et documentation (MSDN) installée localement.
 - Temps : 2h30
-

Pondération (à titre indicatif) :

Question 1 : 1 point
Question 2 : 1 point
Question 3 : 1 point
Question 4 : 1 point
Question 5 : 1 point
Question 6 : 1 point
Question 7 : 2 points
Question 8 : 1 point
Question 9 : 1 point
Question 10 : 2 points
Question 11 : 2 points
Question 12 : 2 points
Question 13 : 2 points
Question 14 : 2 points

Ne pas oublier :

- Lisez attentivement les questions avant de répondre
- Toutes les réponses doivent être stockées sous forme de projets (dont les noms sont précisés ci-dessous) sur votre lecteur G:, où je pourrai les récupérer

Première partie, sans utiliser d'ordinateur

Question 1.

Ecrivez **exactement** ce qui s'affichera à la console si l'on exécute le programme suivant :

```
static void Main(string[] args)
{
    for (int i = 4; i < 20; i = i + 4)
    {
        Console.WriteLine(i);
    }
}
```

Remarques:

- Ce programme ne contient pas d'erreurs
- Inutile de rédiger un texte expliquant ce que le programme fait, indiquez juste le résultat affiché.

Réponse :

Question 2.

Considérons le programme suivant, qui ne contient pas d'erreur :

```
static int f(int x, ref int y, out int z)
{
    x = x + 1;
    y = y * 2;
    z = 17;
    return x;
}
static void Main(string[] args)
{
    int a = 5, b = 6, c = 7;
    int d = f(a, ref b, out c);
    Console.WriteLine(a + " " + b + " " + c + " " + d);
}
```

A la fin de son exécution, quelles seront les valeurs de a, b, c, d (celles qui seront affichées par la dernière instruction) ?

Réponse :

Question 3.

Le programme ci-dessous est censé calculer et afficher le cube de x.

```
static double Cube (double nb)
{
    double res;
    res = nb * nb * nb;
}

static void Main(string[] args)
{
    double x = 3.0, y;
    y = Cube(x);
    Console.WriteLine("Le cube de " + x + " vaut " + y);
}
```

Ce programme comporte cependant une erreur. Corrigez-la en indiquant lisiblement votre correction dans le code ci-dessus (il n'y a qu'une correction à apporter).

Question 4.

Expliquez ce que fait la fonction Test() ci-dessous, qui ne contient pas d'erreur :

```
static bool Test(int a, int b)
{
    bool res;
    if ((a < 0) && (b < 0)) res = true;
    else if ((a >= 0) && (b >= 0)) res = true;
    else res = false;
    return res;
}
```

Remarque : vous ne devez pas décrire ligne par ligne ce que fait le code, mais décrire l'effet global de la fonction (par exemple « cette fonction affiche les nombres de 1 à 10 »).

Réponse :

Deuxième partie, sur PC

L'objectif des questions 5 à 13 est de construire un programme vérifiant si une grille de sudoku est correctement remplie. Chacune des questions ci-dessous vise à écrire un élément contribuant à la solution. La dernière question vise à assembler ces éléments en une solution globale.

Remarques préliminaires :

- Le but de l'examen n'est **pas** d'écrire une interface permettant à un joueur de remplir une grille de sudoku, mais simplement de vérifier si une grille donnée est correctement remplie ou non.
- La vérification d'un sudoku n'est **pas** un processus récursif !

Règles du sudoku

Un jeu de sudoku consiste en une grille de 9x9 cases (Figure 1) que le joueur doit remplir. La grille est organisée en 9 lignes, 9 colonnes et 9 blocs de 3x3 cases, identifiés par les traits en gras sur la figure.

Une fois la grille remplie, le jeu est gagné si toutes les conditions suivantes sont remplies :

- Chaque case est remplie avec un chiffre entre 1 et 9.
- Chaque ligne de la grille contient exactement une fois chacun des chiffres.
- Chaque colonne de la grille contient exactement une fois chacun des chiffres.
- Chaque bloc de la grille contient exactement une fois chacun des chiffres.

Par exemple, le sudoku de la Figure 2 est gagné. En revanche, celui de la Figure 3 ne l'est pas : le 6 apparaît deux fois sur la ligne centrale (et aussi dans le bloc central) et il n'y a pas de 7 dans le bloc central.

	1						5	
9			6	4	1			8
3								6
	6		4		5		9	
	7						3	
8			1		3			4
				8				
2	5						8	7

Figure 1

4	5	2	3	9	1	8	7	6
3	1	8	6	7	5	2	9	4
6	7	9	4	2	8	3	1	5
8	3	1	5	6	4	7	2	9
2	4	5	9	8	7	1	6	3
9	6	7	2	1	3	5	4	8
7	9	6	8	5	2	4	3	1
1	8	3	7	4	9	6	5	2
5	2	4	1	3	6	9	8	7

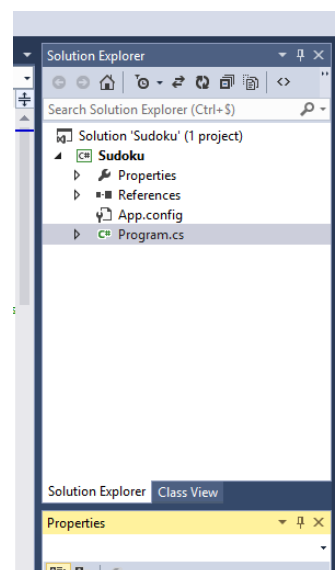
Figure 2

4	5	2	3	9	1	8	7	6
3	1	8	6	7	5	2	9	4
6	7	9	4	2	8	3	1	5
8	3	1	5	6	4	7	2	9
2	4	5	9	8	6	1	6	3
9	6	7	2	1	3	5	4	8
7	9	6	8	5	2	4	3	1
1	8	3	7	4	9	6	5	2
5	2	4	1	3	6	9	8	7

Figure 3

Projet de départ

- Récupérez le projet “Sudoku”, disponible dans le répertoire “K:\IN1-Principes algo. et programm” et copiez-le sur votre lecteur “G:”. Ce projet vous servira de base de travail. C’est également là que vous écrirez vos réponses aux questions ci-dessous.
 - *Remarque : si le code n’apparaît pas lorsque vous ouvrez le projet avec Visual Studio, cliquez sur “Program.cs” dans votre explorateur de solutions (cf. figure ci-contre).*
- Ce projet offre une fonction ChargerGrille() qui reçoit un entier entre 1 et 20 et renvoie un tableau de 9x9 entiers correspondant à une grille de sudoku précalculée. Les grilles 1 à 10 sont correctes, les grilles 11 à 20 sont incorrectes. Il n’est pas nécessaire que vous compreniez comment marche cette fonction : vous pouvez simplement l’utiliser pour charger des grilles à tester.



Questions (suite)

Question 5.

Ecrivez une fonction ChiffreValide() qui reçoit un entier et renvoie une information indiquant si cet entier est compris entre 1 et 9.

Question 6.

Ecrivez une fonction Afficher() qui reçoit un tableau de 9x9 entiers correspondant à une grille de sudoku et l’affiche à l’écran. Il n’est pas nécessaire d’afficher les traits verticaux et horizontaux, ni de traiter des cas particuliers tels qu’une grille incomplète ou de taille incorrecte.

Question 7.

Ecrivez une fonction TraduireCoordonnees() qui reçoit un numéro de bloc et renvoie, en paramètres out, le numéro de ligne et de colonne du coin supérieur gauche de ce bloc. Par exemple, si la fonction reçoit le numéro de bloc 5, elle doit donner en réponse les coordonnées 3, 6 ; pour le bloc numéro 6, elle doit donner 6, 0.

	0	1	2	3	4	5	6	7	8
0	4	5	2	3	9	1	8	7	6
1	3	0	8	6	1	5	2	2	4
2	6	7	9	4	2	8	3	1	5
3	8	3	1	5	6	4	7	2	9
4	2	3	5	9	4	7	1	5	3
5	9	6	7	2	1	3	5	4	8
6	7	9	6	8	5	2	4	3	1
7	1	6	3	7	7	9	6	8	2
8	5	2	4	1	3	6	9	8	7

Figure 4

Question 8.

Ecrivez une fonction `LigneVersListe()` qui reçoit un tableau de 9x9 entiers et un numéro de ligne et renvoie tous les éléments de cette ligne sous la forme d'une liste.

Question 9.

Ecrivez une fonction `ColonneVersListe()` qui reçoit un tableau de 9x9 entiers et un numéro de colonne et renvoie tous les éléments de cette colonne sous la forme d'une liste.

Question 10.

Ecrivez une fonction `BlocVersListe()` qui reçoit un tableau de 9x9 entiers et un numéro de bloc et renvoie tous les éléments de ce bloc sous la forme d'une liste. Vous pouvez bien évidemment réutiliser l'une des fonctions que vous venez d'écrire pour effectuer ce traitement.

Question 11.

Ecrivez une fonction `ListeCorrecte()` qui reçoit une liste de 9 entiers et renvoie une valeur indiquant si cette liste contient bien tous les entiers de 1 à 9, chacun apparaissant exactement une fois. Il n'est pas nécessaire de traiter les cas particuliers tels qu'une liste trop longue ou trop courte. En revanche, la liste pourrait contenir des chiffres non valides (0, par exemple).

Astuce : si la liste contient 9 éléments et que chaque chiffre apparaît au moins une fois, est-il nécessaire de vérifier que certains chiffres n'apparaissent pas en double ?

Question 12.

Ecrivez une fonction `VerifierGrille()` qui reçoit un tableau d'entiers correspondant à une grille de sudoku et renvoie une information indiquant si la grille est correctement remplie. Cette fonction peut bien évidemment utiliser les fonctions ci-dessus.

Question 13.

Ecrivez dans `Main()` le code utilisant la fonction `VerifierGrille()` pour tester les 20 grilles proposées. Pour rappel, les grilles 1 à 10 sont correctes, les grilles 11 à 20 sont incorrectes.

L'objectif de la dernière question, qui n'est pas liée au programme de sudoku ci-dessus, est de montrer votre capacité à utiliser les outils de débogage (*debugging*)

Question 14.

Le projet Facteurs, que vous trouverez dans le répertoire "K:\IN1-Principes algo. et programm", est censé lire un entier au clavier, puis afficher toutes les décompositions possibles de cet entier en produit de deux facteurs. Par exemple, si l'entier entré est 16, le programme doit afficher :

```
16 = 16*1
16 = 8*2
16 = 4*4
16 = 2*8
16 = 1*16
```

Ce programme comporte cependant plusieurs erreurs. Corrigez-le en vous aidant des outils de debugging.